

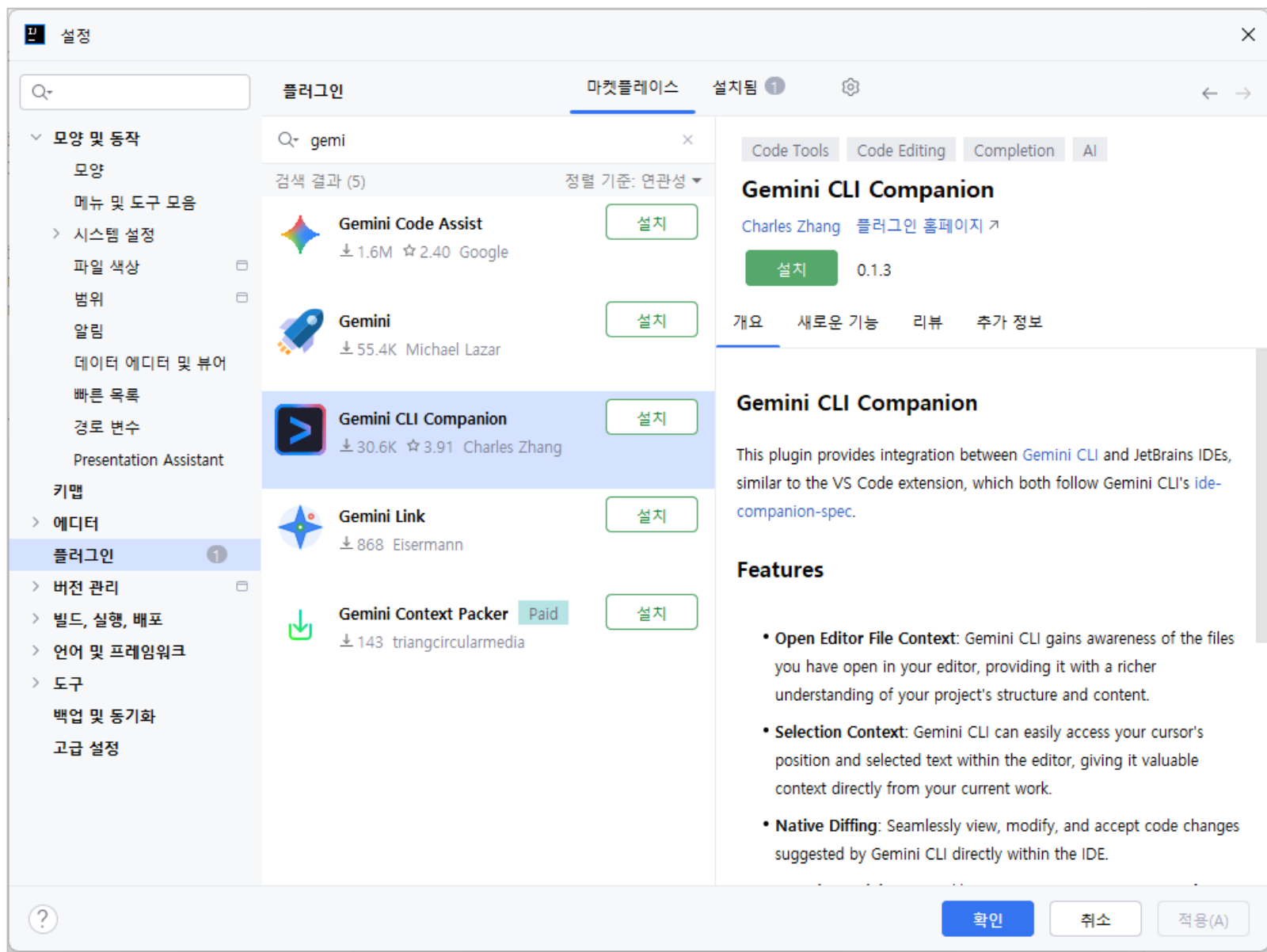
K-디지털 트레이닝

디자인 패턴을 적용한 간단한 Cli 프로그램 개발

[KB] IT's Your Life

✓ IntelliJ에서 Gemini CLI 사용시 주의 사항

- Gemini CLI를 사용하여 파일을 생성했을 때 프로젝트 창에 즉시 나타나지 않는 현상
 - IDE의 가상 파일 시스템(VFS)이 외부(터미널)에서 발생한 변화를 실시간으로 감지하지 못해서 발생
- 해결방법
 - Synchronize (동기화) 단축키 활용 (가장 빠름)
 - Windows / Linux: Ctrl + Alt + Y
 - macOS: Option + Command + Y
 - Gemini CLI 전용 'IDE Companion' 활용 (권장)
 - IntelliJ 플러그인 마켓플레이스에서 "**Gemini CLI Companion**"을 검색하여 설치
 - IntelliJ 내장 터미널에서 Gemini CLI를 실행한 후 다음 명령어를 입력
/ide enable

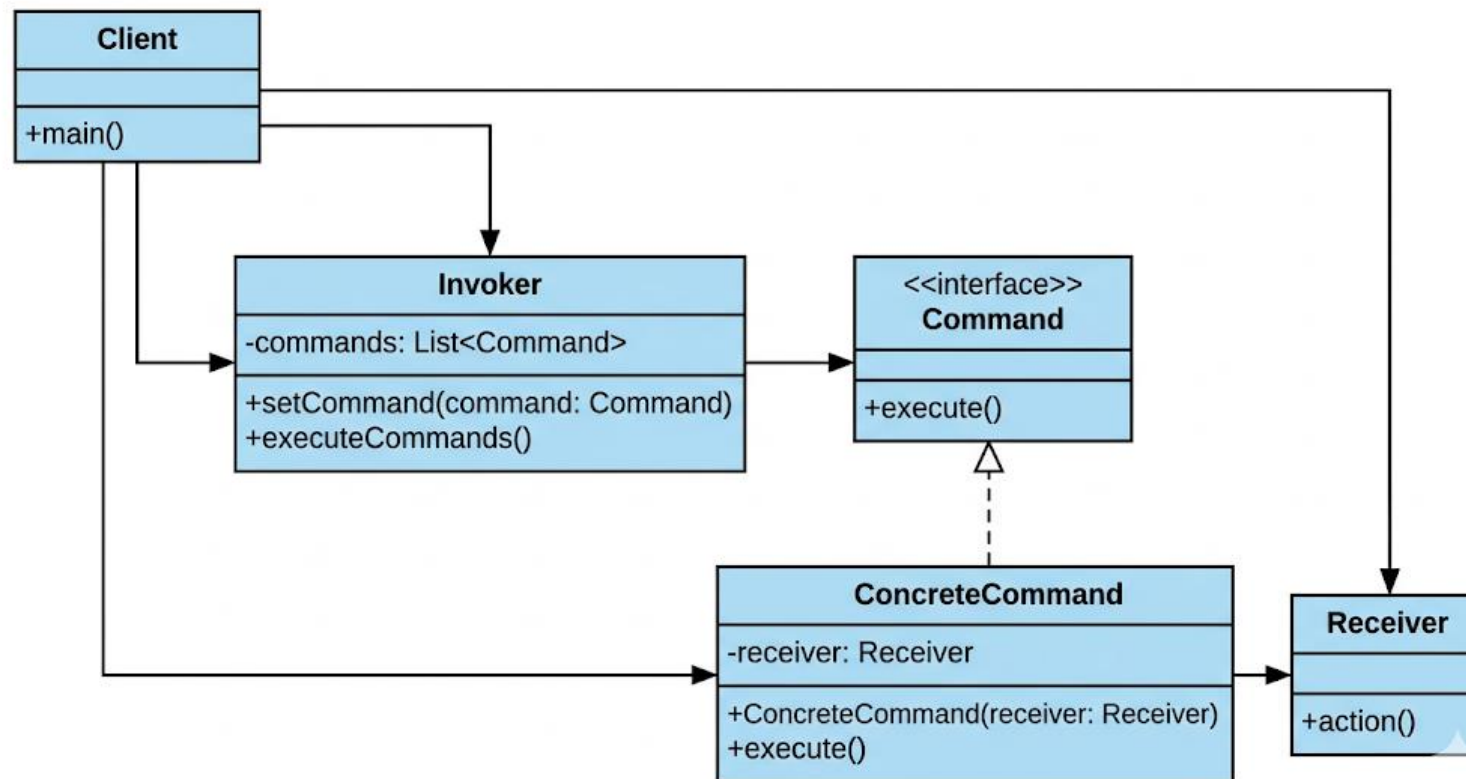


✓ Command 패턴

- 요청을 객체로 캡슐화하여, 사용자(Client)가 다양한 요청을 동일한 방식으로 처리할 수 있도록 하는 행동 디자인 패턴입
- 요청을 큐(Queue)에 저장하거나, 로깅(Logging), 취소(Undo) 기능을 구현하는 데 사용

✓ Command 패턴

○ Class Diagram



✓ Client (클라이언트)

○ 역할

- 패턴을 사용하는 주체
- Invoker, ConcreteCommand, Receiver 객체를 생성하고 서로 연결하는 역할

○ 책임

- Invoker, ConcreteCommand, Receiver 객체 생성 및 초기화
- ConcreteCommand 객체를 생성할 때 Receiver 객체를 전달하여 연결
- Invoker 객체에 ConcreteCommand 객체를 등록

✓ Invoker (호출자)

○ 역할

- 명령(Command)을 실행하는 역할
- 명령 객체를 저장하고, 요청이 들어오면 해당 명령의 execute() 메서드를 호출

○ 책임

- 명령 객체를 저장하기 위한 컬렉션(예: List) 관리
- setCommand() 메서드를 통해 명령 객체를 등록
- executeCommands() 메서드를 통해 저장된 명령들을 순서대로 실행 (또는 특정 조건에 따라 실행)

✓ Command (명령 인터페이스)

- 역할
 - 모든 구체적인 명령 클래스들이 구현해야 하는 공통 인터페이스
 - `execute()` 메서드를 정의
- 책임
 - 모든 명령에 공통적인 인터페이스 제공

✓ ConcreteCommand (구체적인 명령 클래스)

- 역할
 - Command 인터페이스를 구현하며, 실제 요청을 처리하는 `execute()` 메서드를 정의
 - Receiver 객체를 참조하여 실제 작업을 수행
- 책임
 - Command 인터페이스 구현
 - Receiver 객체를 참조하여 실제 작업 수행
 - 필요에 따라 명령 실행 전후의 부가적인 작업 수행 (예: 로깅, 취소 기능 구현을 위한 상태 저장)

✓ Receiver (수신자)

- 역할
 - 실제 요청을 처리하는 객체
 - ConcreteCommand 객체가 참조하여 작업을 수행
- 책임
 - 실제 비즈니스 로직 수행요청에 대한 결과 반환 (필요에 따라)

파일명

```
// Receiver 클래스
class Light {
    public void turnOn() {
        System.out.println("전등을 켭니다.");
    }

    public void turnOff() {
        System.out.println("전등을 끕니다.");
    }
}

// Command 인터페이스
interface Command {
    void execute();
}
```


파일명

```
// ConcreteCommand 클래스 (전등 켜기 명령)
class LightOnCommand implements Command {
    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOn();
    }
}

// ConcreteCommand 클래스 (전등 끄기 명령)
class LightOffCommand implements Command {
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOff();
    }
}
```

파일명

```
// Invoker 클래스 (리모컨)
class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        if (command != null) {
            command.execute();
        }
    }
}
```

파일명

```
// Client 클래스
public class Client {
    public static void main(String[] args) {
        Light light = new Light();
        Command lightOnCommand = new LightOnCommand(light);
        Command lightOffCommand = new LightOffCommand(light);

        RemoteControl remoteControl = new RemoteControl();

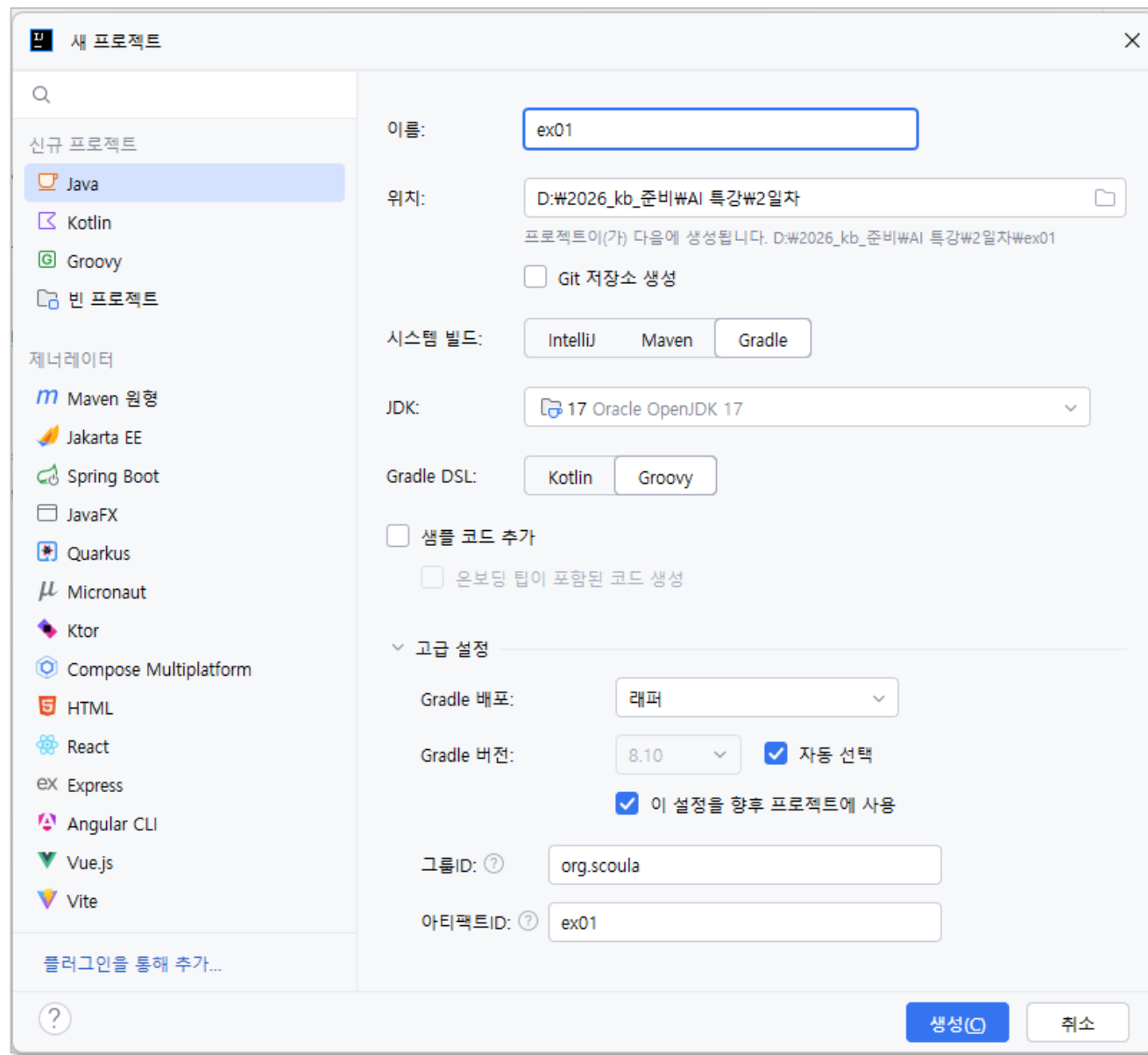
        remoteControl.setCommand(lightOnCommand);
        remoteControl.pressButton(); // 전등을 켭니다.

        remoteControl.setCommand(lightOffCommand);
        remoteControl.pressButton(); // 전등을 끕니다.
    }
}
```

✓ Travel 어플리케이션

- Command 패턴 기반의 CLI 어플리케이션
- travel.csv에 관광지 정보 저장
- travel.csv 파일을 읽어 관광지 검색/조회

✓ 프로젝트 만들기



The image shows the 'New Project' dialog box in IntelliJ IDEA. The left sidebar lists various project types under 'New Project' and 'Generators'. 'Java' is selected under 'New Project'. The main area contains configuration fields for the new project.

새 프로젝트

이름: ex01

위치: D:\2026_kb_준비\AI 특강\2일차
프로젝트이(가) 다음에 생성됩니다. D:\2026_kb_준비\AI 특강\2일차\ex01

☐ Git 저장소 생성

시스템 빌드: IntelliJ Maven Gradle

JDK: 17 Oracle OpenJDK 17

Gradle DSL: Kotlin Groovy

☐ 샘플 코드 추가
☐ 온보딩 팁이 포함된 코드 생성

고급 설정

Gradle 배포: 래퍼

Gradle 버전: 8.10 ☒ 자동 선택

☒ 이 설정을 향후 프로젝트에 사용

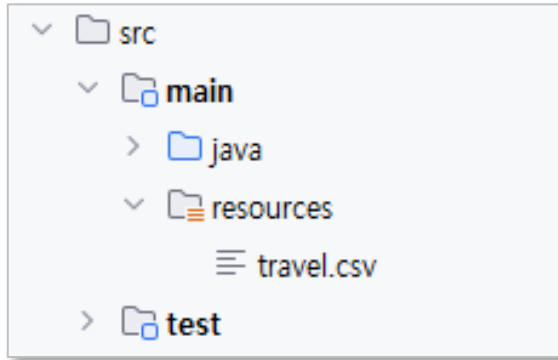
그룹ID: org.scoula

아티팩트ID: ex01

생성(C) 취소

✓ 준비

- travel.csv 파일을 resources 폴더에 배치



- **프롬프트**

Lombok 라이브러리를 추가해줘.

csv 파일을 처리하기위한 OpenCSV 라이브러리 추가해줘.

cf) gradle 동기화 실행

Gemini.md

```
# Project Overview: Java Cli Application
- travel.csv 파일을 읽어서 여행지 정보를 관리하는 CLI 애플리케이션
- 사용자는 여행지를 추가, 삭제, 조회(검색)할 수 있음
- 여행지 정보를 파일에 저장하고 불러올 수 있음

## Tech Stack
- Java 17
- Lombok
- JUnit 5
- Gradle

## Project Structure
- base package: org.scoula
- design pattern: Command Pattern
- main class: Main.java
- command interface: Command.java
```

✓ Command 패턴기반 CLI 프레임워크 만들기

○ 프롬프트

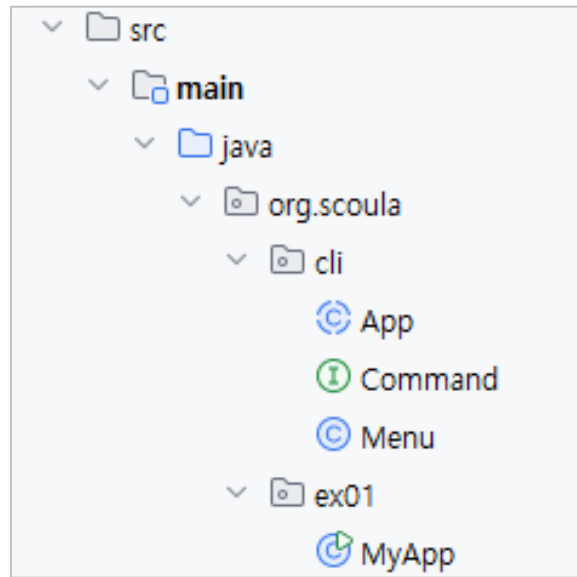
Command 패턴을 기반으로 한 CLI Application 프레임워크를 만들어줘.
org.scoula.cli 패키지에 만들어줘.
명령 선택은 명령 목록을 출력한 후 인덱스 선택으로 실행하도록 해줘.

주요 설계 내용

1. **Command 인터페이스**: 모든 실행 가능한 로직을 `execute()` 메소드로 통일합니다.
2. **Menu & MenuItem 클래스**:
 - **MenuItem**: 메뉴 제목과 연결된 **Command** 객체를 관리하며, Lombok의 `@Getter`와 `@AllArgsConstructor`를 사용하여 코드를 최소화했습니다.
 - **Menu**: **MenuItem**의 리스트를 관리하며, `print()`를 통해 사용자에게 인덱스 번호 (1부터 시작)와 함께 메뉴를 보여줍니다.
3. **App 클래스**:
 - 사용자의 숫자 입력을 받아 해당 인덱스의 명령을 찾아 실행하는 메인 루프(`run()`)를 포함합니다.
 - 입력값이 숫자가 아니거나 범위를 벗어날 경우에 대한 예외 처리가 되어 있습니다.

실행 예시

- 메인 메소드에서 `app.register("메뉴명", () -> { 로직 });` 형태로 람다식이나 익명 객체를 사용해 간편하게 명령어를 등록할 수 있습니다.
- 현재 "인사", "날짜", "종료" 명령어가 기본 예시로 등록되어 있습니다.



org.scoula.cli.Command.java

```
package org.scoula.cli;  
  
public interface Command {  
    void execute();  
}
```

org.scoula.cli.Menu.java

```
package org.scoula.cli;

import lombok.AllArgsConstructor;
import lombok.Getter;

import java.util.ArrayList;
import java.util.List;

@Getter
public class Menu {
    private final List<MenuItem> items = new ArrayList<>();

    public void add(MenuItem item) {
        items.add(item);
    }

    public void print() {
        System.out.println("-----");
        for (int i = 0; i < items.size(); i++) {
            System.out.printf("%d. %s | ", i + 1, items.get(i).getTitle());
        }
        System.out.println();
        System.out.println("-----");
    }
}
```

org.scoula.cli.Menu.java

```
public Command getCommand(int index) {
    if (index > 0 && index <= items.size()) {
        return items.get(index - 1).getCommand();
    }
    return null;
}

@Getter
@AllArgsConstructor
public static class MenuItem {
    private final String title;
    private final Command command;
}
}
```

org.scoula.cli.App.java

```
package org.scoula.cli;

import java.util.Scanner;

public abstract class App {
    protected final Menu menu = new Menu();

    public void register(String title, Command command) {
        menu.add(new Menu.MenuItem(title, command));
    }

    public abstract void init(); // Subclasses will override this to register commands
```

org.scoula.cli.App.java

```
public void run() {
    init();
    Scanner scanner = new Scanner(System.in);
    while (true) {
        menu.print();
        System.out.print("선택: ");
        String input = scanner.nextLine();
        try {
            int index = Integer.parseInt(input);
            Command command = menu.getCommand(index);
            if (command != null) {
                command.execute();
            } else {
                System.out.println("잘못된 입력입니다. 메뉴 범위를 확인하세요.");
            }
        } catch (NumberFormatException e) {
            System.out.println("숫자를 입력해주세요.");
        }
    }
}
```

org.scoula.ex01.MyApp

```
package org.scoula.ex01;

import org.scoula.cli.App;

public class MyApp extends App {
    @Override
    public void init() {
        register("인사", () -> System.out.println("안녕하세요!"));
        register("날짜", () -> System.out.println("현재 날짜: " + java.time.LocalDate.now()));
        register("종료", () -> {
            System.out.println("프로그램을 종료합니다.");
            System.exit(0);
        });
    }

    public static void main(String[] args) {
        new MyApp().run();
    }
}
```

✓ 실행

1. 인사 | 2. 날짜 | 3. 종료 |

선택: 1
안녕하세요!

1. 인사 | 2. 날짜 | 3. 종료 |

선택: 2
현재 날짜: 2026-04-13

1. 인사 | 2. 날짜 | 3. 종료 |

선택: 6
잘못된 입력입니다. 메뉴 범위를 확인하세요.

1. 인사 | 2. 날짜 | 3. 종료 |

선택: 3
프로그램을 종료합니다.

✓ 클래스 다이어그램 그리기

○ 프롬프트

org.scoula.cli 패키지와 org.scoula.ex01 패키지를 클래스 다이어그램으로 그려서 cli_class_diagram.md 파일로 만들어 줘.

cli_class_diagram.md

CLI Framework and Example Application Class Diagram

```
```mermaid
classDiagram
 package org.scoula.cli {
 class Command {
 <<interface>>
 +execute() void
 }

 class Menu {
 -items: List~MenuItem~
 +add(MenuItem) void
 +print() void
 +getCommand(int index) Command
 }

 class MenuItem {
 -title: String
 -command: Command
 +getTitle() String
 +getCommand() Command
 }
 }
```
```

cli_class_diagram.md

```
...
%% Relationships
App "1" *-- "1" Menu : contains
Menu "1" *-- "n" MenuItem : contains
MenuItem "1" o-- "1" Command : references
MyApp --|> App : extends
...
```

다이어그램 설명

1. ****`org.scoula.cli` (Framework Core)**:**
 - ****`Command`**:** 실행 로직을 `execute()` 메서드로 규격화한 인터페이스입니다.
 - ****`App`**:** CLI 어플리케이션의 핵심 루프와 메뉴 관리를 담당하는 추상 클래스입니다.
 - ****`Menu` & `MenuItem`**:** 등록된 명령어를 제목과 함께 관리하고 출력하는 역할을 합니다.
2. ****`org.scoula.ex01` (Implementation Example)**:**
 - ****`MyApp`**:** 프레임워크인 `App`` 클래스를 상속받아 실제 동작을 구현한 예시 클래스입니다. `init()` 메서드를 오버라이드하여 사용할 명령어들을 등록합니다.

✓ 이미지로 그리기

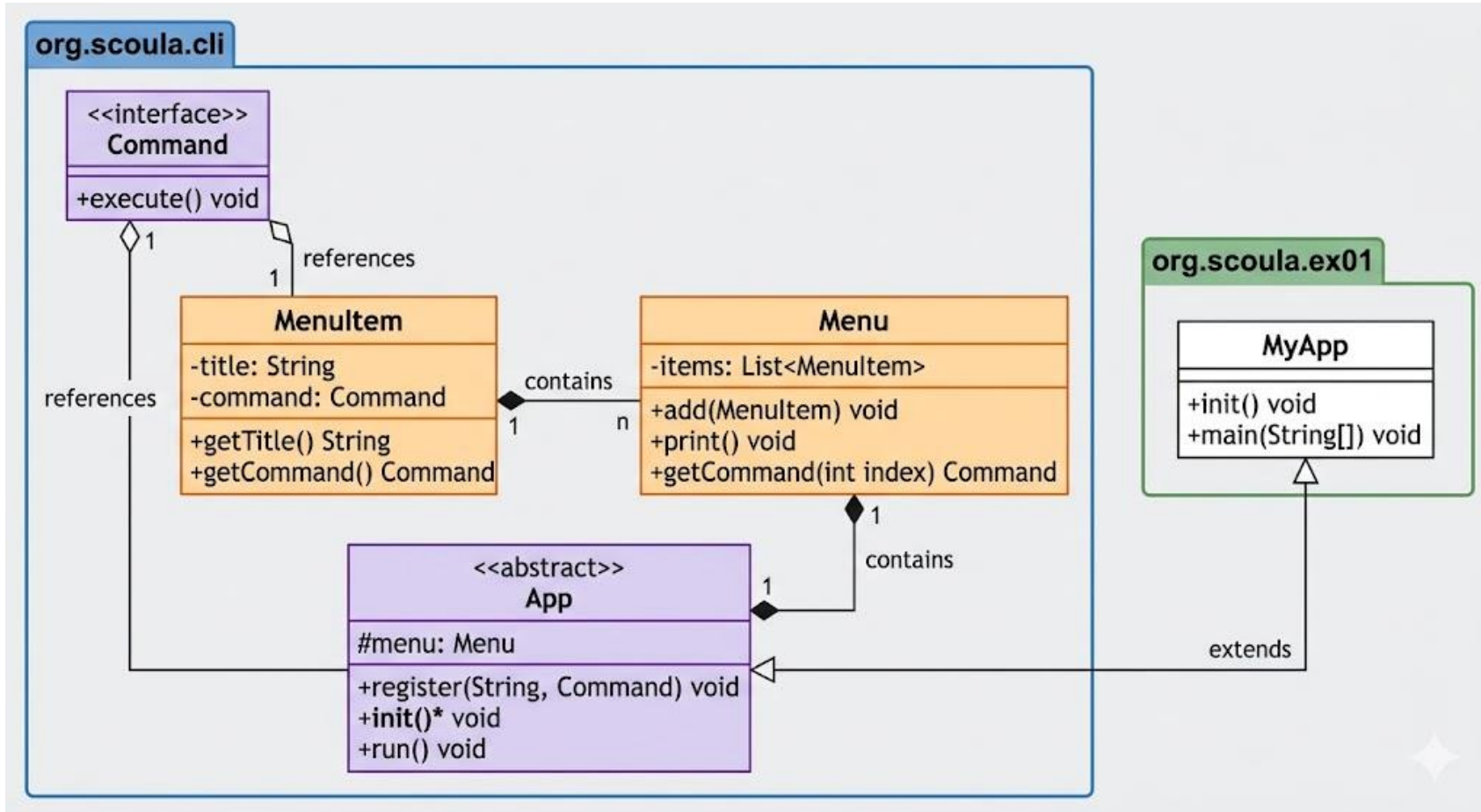
- <https://gemini.google.com/>

- 프로프트

```
```mermaid 코드
```

```
...
```
```

클래스 다이어그램을 이미지로 그려주고, 설명해줘.



✓ Travel 도메인 클래스 생성

○ 프롬프트

travel.csv 파일에 맞는 Travel 클래스를 org.scoula.travel.domain 패키지에 만들어줘.

org.scoula.travel.domain.Travel.java

```
package org.scoula.travel.domain;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Travel {
    private int no;
    private String district;
    private String title;
    private String description;
    private String address;
    private String phone;
}
```

✓ TravelDao 클래스 생성

○ 프롬프트

어플리케이션이 시작될 때 travel.csv 파일을 읽어 Travel 목록을 준비하고, 이에 대한 CRUD를 수행하는 Dao 클래스를 만들어줘.

주요 구현 내용

1. CSV 데이터 로딩: OpenCSV의 `CsvToBeanBuilder`를 사용하여 `travel.csv` 파일을 읽어와 `List<Travel>`로 자동 매핑합니다. (UTF-8 인코딩 처리 포함)
2. CRUD 기능:
 - `findAll()`: 모든 여행 정보 목록을 반환합니다.
 - `findByNo(int no)`: 특정 번호의 여행 정보를 조회합니다 (`Optional` 사용).
 - `create(Travel travel)`: 새로운 여행 정보를 추가하며, 번호(`no`)를 자동으로 할당합니다.
 - `update(Travel travel)`: 기존 정보를 수정합니다.
 - `delete(int no)`: 특정 번호의 정보를 삭제합니다.
3. 메모리 기반: 로딩된 데이터는 `ArrayList`에 저장되어 실행 중 메모리에서 관리됩니다.

org.scoula.travel.domain.Travel.java

```
package org.scoula.travel.domain;

import com.opencsv.bean.CsvBindByName;
...
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Travel {
    @CsvBindByName(column = "no")
    private int no;

    @CsvBindByName(column = "district")
    private String district;

    @CsvBindByName(column = "title")
    private String title;

    @CsvBindByName(column = "description")
    private String description;

    @CsvBindByName(column = "address")
    private String address;

    @CsvBindByName(column = "phone")
    private String phone;
}
```

org.scoula.travel.dao.TravelDao.java

```
package org.scoula.travel.dao;

import com.opencsv.bean.CsvToBeanBuilder;
import org.scoula.travel.domain.Travel;

import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class TravelDao {
    private List<Travel> travelList = new ArrayList<>();

    public TravelDao() {
        init();
    }
}
```

org.scoula.travel.dao.TravelDao.java

```
private void init() {
    try (InputStreamReader reader = new InputStreamReader(
        getClass().getClassLoader().getResourceAsStream("travel.csv"),
        StandardCharsets.UTF_8)) {

        travelList = new CsvToBeanBuilder<Travel>(reader)
            .withType(Travel.class)
            .build()
            .parse();

        System.out.println("CSV 로드 완료: " + travelList.size() + "개 항목");
    } catch (Exception e) {
        System.err.println("CSV 파일을 읽는 중 오류 발생: " + e.getMessage());
        travelList = new ArrayList<>();
    }
}

// CRUD: Read All
public List<Travel> findAll() {
    return new ArrayList<>(travelList);
}
```

org.scoula.travel.dao.TravelDao.java

```
// CRUD: Read One by No
public Optional<Travel> findByNo(int no) {
    return travelList.stream()
        .filter(t -> t.getNo() == no)
        .findFirst();
}

// CRUD: Create
public void create(Travel travel) {
    // 자동 번호 할당 (최대값 + 1)
    int maxNo = travelList.stream()
        .mapToInt(Travel::getNo)
        .max()
        .orElse(0);
    travel.setNo(maxNo + 1);
    travelList.add(travel);
}
```

org.scoula.travel.dao.TravelDao.java

```
// CRUD: Update
public boolean update(Travel travel) {
    Optional<Travel> existing = findByNo(travel.getNo());
    if (existing.isPresent()) {
        int index = travellist.indexOf(existing.get());
        travellist.set(index, travel);
        return true;
    }
    return false;
}

// CRUD: Delete
public boolean delete(int no) {
    return travellist.removeIf(t -> t.getNo() == no);
}
}
```

✓ TravelDao 클래스 생성

○ 수정 프롬프트

TravelDao에 페이지네이션을 위한 getPage 메서드를 추가해줘. 한 페이지당 10개씩 담아줘.

✦ TravelDao에 페이지네이션을 지원하는 `getPage(int page)` 메서드와 전체 페이지 수를 반환하는 `getTotalPages()` 메서드를 추가했습니다.

- `getPage(int page)`: 요청한 페이지 번호(1부터 시작)에 해당하는 10개의 여행 정보 목록을 반환합니다. 페이지 범위를 벗어날 경우 빈 리스트를 반환하며, 마지막 페이지의 경우 남은 데이터만큼만 반환하도록 처리했습니다.
- `getTotalPages()`: 전체 데이터 건수를 기준으로 총 페이지 수를 계산하여 반환합니다.

org.scoula.travel.dao.TravelDao.java

```
// Pagination: Get a sublist of 10 items
public List<Travel> getPage(int page) {
    int pageSize = 10;
    int totalItems = travellist.size();
    int totalPages = (int) Math.ceil((double) totalItems / pageSize);

    if (page < 1 || page > totalPages) {
        return new ArrayList<>();
    }

    int fromIndex = (page - 1) * pageSize;
    int toIndex = Math.min(fromIndex + pageSize, totalItems);

    return new ArrayList<>(travellist.subList(fromIndex, toIndex));
}

public int getTotalPages() {
    return (int) Math.ceil((double) travellist.size() / 10);
}
```

✓ TravelDao 클래스 생성

○ 수정 프롬프트

TravelDao에 특정 권역(district)에 대한 Travel 목록을 리턴하는 메서드를 추가해줘.

✦ TravelDao에 권역별 조회를 위한 `findByDistrict(String district)` 메서드를 추가했습니다. 이 메서드는 대소문자 구분 없이(`equalsIgnoreCase`) 입력받은 권역명과 일치하는 여행 정보 목록을 반환합니다.

org.scoula.travel.dao.TravelDao.java

```
...
// Search by District
public List<Travel> findByDistrict(String district) {
    return travellist.stream()
        .filter(t -> t.getDistrict().equalsIgnoreCase(district))
        .toList();
}
...
```

✓ TravelDao 클래스 생성

○ 수정 프롬프트

TravelDao에 제목(title)과 설명(description)에 대한 키워드 검색 메서드를 추가해줘.

◆ TravelDao에 제목(title)과 설명(description)을 대상으로 하는 키워드 검색 메서드 `search(String keyword)`를 추가했습니다. 대소문자를 구분하지 않고 키워드가 포함된 모든 여행 정보를 반환합니다..

org.scoula.travel.dao.TravelDao.java

```
...
// Keyword Search in title and description
public List<Travel> search(String keyword) {
    String lowerKeyword = keyword.toLowerCase();
    return travelList.stream()
        .filter(t -> t.getTitle().toLowerCase().contains(lowerKeyword) ||
                    t.getDescription().toLowerCase().contains(lowerKeyword))
        .toList();
}
...
```

✓ 어플리케이션 구성

○ 프롬프트

다음과 같이 메뉴를 구성 하고, TravelService에 해당 메뉴를 실행하는 메서드를 연결해줘.

1. 전체 목록
2. 페이지별 목록
3. 권역별 목록
4. 상세보기
5. 종료

실제 기능 구현은 아직 하지마.

✦ 요청하신 대로 메뉴 구성과 TravelService의 메서드 연결을 완료했습니다.

주요 구성 내용

1. TravelService 클래스: 각 메뉴 항목에 대응하는 스텝(Stub) 메서드(showAll, showPage, showByDistrict, showDetail, exit)를 생성했습니다.
2. TravelApp 클래스: org.scoula.cli.App을 상속받아 init() 메서드에서 TravelService의 메서드들을 메뉴로 등록했습니다.
 - 1. 전체 목록
 - 2. 페이지별 목록
 - 3. 권역별 목록
 - 4. 상세보기
 - 5. 종료

현재는 각 메뉴 선택 시 "미구현" 메시지가 출력되도록 설정되어 있으며, 이후 요구사항에 따라 실제 기능을 구현할 준비가 되었습니다..

org.scoula.travel.service.TravelService.java

```
package org.scoula.travel.service;

import org.scoula.travel.dao.TravelDao;

public class TravelService {
    private final TravelDao travelDao = new TravelDao();

    public void showAll() {
        System.out.println("1. 전체 목록 실행 (미구현)");
    }

    public void showPage() {
        System.out.println("2. 페이지별 목록 실행 (미구현)");
    }

    public void showByDistrict() {
        System.out.println("3. 권역별 목록 실행 (미구현)");
    }

    public void showDetail() {
        System.out.println("4. 상세보기 실행 (미구현)");
    }
}
```

org.scoula.travel.service.TravelService.java

```
public void exit() {  
    System.out.println("5. 프로그램을 종료합니다.");  
    System.exit(0);  
}  
}
```

org.scoula.travel.TravelApp.java

```
package org.scoula.travel;

import org.scoula.cli.App;
import org.scoula.travel.service.TravelService;

public class TravelApp extends App {
    private final TravelService travelService = new TravelService();

    @Override
    public void init() {
        register("전체 목록", travelService::showAll);
        register("페이지별 목록", travelService::showPage);
        register("권역별 목록", travelService::showByDistrict);
        register("상세보기", travelService::showDetail);
        register("종료", travelService::exit);
    }

    public static void main(String[] args) {
        new TravelApp().run();
    }
}
```

✓ TravelApp 실행

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택: 1

1. 전체 목록 실행 (미구현)

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택: 2

2. 페이지별 목록 실행 (미구현)

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택:

✓ 전체 목록 보기 구현

○ 프롬프트

전체 목록 보기 기능을 구현해줘.

✦ `TravelService`의 `showAll()` 메서드에 전체 목록 보기 기능을 구현했습니다.

- 동작 방식: `travelDao.findAll()`을 통해 모든 여행 정보를 가져온 뒤, 번호, 권역, 제목, 주소, 전화번호를 포맷에 맞춰 출력합니다.
- 데이터 원천: 애플리케이션 시작 시 `travel.csv`로부터 로드된 데이터를 사용합니다.

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택: 1

[전체 목록 보기]

번호 | 권역 | 제목 | 주소 | 전화번호

1 | 수도권 | 인천 개항장역사문화공간 | 인천광역시 중구 제물량로232번안길 27 | 인천 중구 문화유산팀 032-760-6448
2 | 수도권 | 인천 월미문화의거리 | 인천광역시 중구 월미로 252 | 032-764-0842
3 | 수도권 | 인천 연안부두해양광장 | 인천광역시 중구 연안부두로 70 (항동7가) | 032-777-1330
4 | 수도권 | 인천 하나개해수욕장 | 인천광역시 중구 무의동 산189번지 (무의동) | 032-751-8833
5 | 수도권 | 양평 세미원 | 경기도 양평군 양서면 양수로 93 | 031-775-1835

org.scoula.travel.service.TravelService.java

```
package org.scoula.travel.service;

import org.scoula.travel.dao.TravelDao;

public class TravelService {
    private final TravelDao travelDao = new TravelDao();

    public void showAll() {
        System.out.println("\n[전체 목록 보기]");
        System.out.println("번호 | 권역 | 제목 | 주소 | 전화번호");
        System.out.println("-----");
        for (org.scoula.travel.domain.Travel travel : travelDao.findAll()) {
            System.out.printf("%d | %s | %s | %s | %s\n",
                travel.getNo(), travel.getDistrict(), travel.getTitle(),
                travel.getAddress(), travel.getPhone());
        }
        System.out.println("-----");
    }
}
```

✓ 전체 목록 보기 구현

○ 수정 프롬프트

목록 보기를 권역(district)/제목(title) 순으로 정렬해서 보여줘.

cf) Dao 설계 시 누락된 부분 !!

✦ 전체 목록 보기 시 권역(District) → 제목(Title) 순으로 정렬하여 출력되도록 수정했습니다.

- 동작 방식: Java Stream API의 `sorted()`와 `Comparator.comparing(...).thenComparing(...)`을 사용하여 다중 정렬을 수행합니다.
- 정렬 순서: 먼저 권역별로 묶어 보여주고, 같은 권역 내에서는 제목의 가나다순으로 정렬됩니다.

[전체 목록 보기 - 정렬: 권역/제목]

번호 | 권역 | 제목 | 주소 | 전화번호

```
-----
101 | 강원권 | 강릉 경포 해변 | 강원도 강릉시 창해로 514 (안현동) | 033-640-4901
105 | 강원권 | 강릉 솔향수목원 | 강원도 강릉시 구정면 수목원길 156 | 033-660-2322
100 | 강원권 | 강릉 연곡해변캠핑장 | 강원도 강릉시 연곡면 해안로 1282 | 033-662-2900
106 | 강원권 | 강릉 정동진 모래시계공원 | 강원도 강릉시 강동면 현화로 990-1 | 033-640-4533
103 | 강원권 | 강릉 커피거리 | 강원도 강릉시 창해로14번길 20-1 | 033-640-4537강릉시 관광안내소 033-640-4531
104 | 강원권 | 강릉 통일공원 | 강원도 강릉시 강동면 울곡로 1616 | 강릉통일공원 안보전시관 033-640-4469
102 | 강원권 | 강릉 허균?허난설헌 기념공원 | 강원도 강릉시 난설헌로193번길 1-16 (초당동) | 033-640-5119
107 | 강원권 | 동해 망상해수욕장 | 강원도 동해시 동해대로 6270-10 (망상동) | 033-530-2800
```

org.scoula.travel.service.TravelService.java

```
...
public void showAll() {
    System.out.println("\n[전체 목록 보기 - 정렬: 권역/제목]");
    System.out.println("번호 | 권역 | 제목 | 주소 | 전화번호");
    System.out.println("-----");

    travelDao.findAll().stream()
        .sorted(java.util.Comparator.comparing(org.scoula.travel.domain.Travel::getDistrict)
            .thenComparing(org.scoula.travel.domain.Travel::getTitle))
        .forEach(travel -> System.out.printf("%d | %s | %s | %s | %s\n",
            travel.getNo(), travel.getDistrict(), travel.getTitle(),
            travel.getAddress(), travel.getPhone()));

    System.out.println("-----");
}
...
```

✓ 페이지별 목록 보기 기능 구현

○ 프롬프트

페이지별 목록 보기를 권역(district)/제목(title) 순으로 정렬해서 보여줘.

- 처음에는 1페이지를 보여주고,
- 다음에는 사용자로부터 보고자 하는 페이지 번호를 입력 받아 보여줘.
- 그냥 엔터를 치면 페이지별 목록 보기를 마치고 메인 메뉴로 돌아가줘.
- "현재 페이지 번호/전체 페이지 수" 정보를 항상 보여줘.

✦ `TravelService`의 `showPage()` 메서드에 페이지별 목록 보기 기능을 구현했습니다.

구현 사항

1. 정렬: 전체 데이터를 먼저 **권역(District) → 제목(Title)** 순으로 정렬한 뒤 페이지를 나눕니다.
2. 초기 화면: 시작 시 자동으로 **1페이지**를 출력합니다.
3. 사용자 상호작용:
 - 페이지 하단에 **"현재 페이지 / 전체 페이지"** 정보를 표시합니다.
 - 이동하고 싶은 페이지 번호를 입력받아 즉시 해당 페이지로 이동합니다.
 - **엔터(Empty Input)**를 치면 루프를 빠져나와 메인 메뉴로 돌아갑니다.
4. 예외 처리: 숫자가 아닌 입력이나 범위를 벗어난 페이지 번호에 대한 안내 메시지를 출력합니다.

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택: 2

[페이지별 목록 보기]

번호 | 권역 | 제목 | 주소 | 전화번호

| | | | | |
|-----|-----|-----------------|------------------------------|------------------------------------|
| 101 | 강원권 | 강릉 경포 해변 | 강원도 강릉시 창해로 514 (안현동) | 033-640-4901 |
| 105 | 강원권 | 강릉 솔향수목원 | 강원도 강릉시 구정면 수목원길 156 | 033-660-2322 |
| 100 | 강원권 | 강릉 연곡해변캠핑장 | 강원도 강릉시 연곡면 해안로 1282 | 033-662-2900 |
| 106 | 강원권 | 강릉 정동진 모래시계공원 | 강원도 강릉시 강동면 헌화로 990-1 | 033-640-4533 |
| 103 | 강원권 | 강릉 커피거리 | 강원도 강릉시 창해로14번길 20-1 | 033-640-4537강릉시 관광안내소 033-640-4531 |
| 104 | 강원권 | 강릉 통일공원 | 강원도 강릉시 강동면 울곡로 1616 | 강릉통일공원 안보전시관 033-640-4469 |
| 102 | 강원권 | 강릉 허균?허난설헌 기념공원 | 강원도 강릉시 난설헌로193번길 1-16 (초당동) | 033-640-5119 |
| 107 | 강원권 | 동해 망상해수욕장 | 강원도 동해시 동해대로 6270-10 (망상동) | 033-530-2800 |
| 98 | 강원권 | 속초 아바이마을 | 강원도 속초시 청호로 122-1 | 033-639-2690 |
| 99 | 강원권 | 속초 해수욕장관광지 | 강원도 속초시 해오름로 190 (조양동) | 033-639-2027 |

현재 페이지: 1 / 12 (전체 항목: 112)

이동할 페이지 번호 (종료하려면 Enter):

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택:

org.scoula.travel.service.TravelService.java

```
...
public void showPage() {
    List<org.scoula.travel.domain.Travel> sortedList = travelDao.findAll().stream()
        .sorted(java.util.Comparator.comparing(org.scoula.travel.domain.Travel::getDistrict)
            .thenComparing(org.scoula.travel.domain.Travel::getTitle))
        .toList();

    int pageSize = 10;
    int totalItems = sortedList.size();
    int totalPages = (int) Math.ceil((double) totalItems / pageSize);
    int currentPage = 1;

    while (true) {
        int fromIndex = (currentPage - 1) * pageSize;
        int toIndex = Math.min(fromIndex + pageSize, totalItems);
        List<org.scoula.travel.domain.Travel> pageItems = sortedList.subList(fromIndex, toIndex);

        System.out.println("\n[페이지별 목록 보기]");
        System.out.println("번호 | 권역 | 제목 | 주소 | 전화번호");
        System.out.println("-----");
        for (org.scoula.travel.domain.Travel travel : pageItems) {
            System.out.printf("%d | %s | %s | %s | %s\n",
                travel.getNo(), travel.getDistrict(), travel.getTitle(),
                travel.getAddress(), travel.getPhone());
        }
        System.out.println("-----");
    }
}
```

org.scoula.travel.service.TravelService.java

```
System.out.printf("현재 페이지: %d / %d (전체 항목: %d)%n", currentPage, totalPages, totalItems);

System.out.print("이동할 페이지 번호 (종료하려면 Enter): ");
String input = scanner.nextLine().trim();

if (input.isEmpty()) {
    break;
}

try {
    int page = Integer.parseInt(input);
    if (page >= 1 && page <= totalPages) {
        currentPage = page;
    } else {
        System.out.printf("1에서 %d 사이의 페이지 번호를 입력해주세요.%n", totalPages);
    }
} catch (NumberFormatException e) {
    System.out.println("숫자를 입력하거나 Enter를 눌러주세요.");
}

}

...
}
```


✓ 페이지별 목록 보기 기능 구현

○ 프롬프트

다음과 같이 권역별 목록 보기를 제목(title) 순으로 정렬해서 보여줘.

- 번호) 권역 목록을 먼저 출력
- 해당 번호를 입력하면 해당 권역의 목록을 출력

✦ `TravelDao`에 중복 없는 권역 목록을 가져오는 기능을 추가하고, `TravelService`에서 이를 활용하여 **2단계 권역 선택** 기능을 구현했습니다.

구현 사항

1. **권역 목록 출력**: 사용 가능한 모든 권역을 **1) 수도권 2) 강원권 ...** 과 같이 번호를 매겨 먼저 보여줍니다.
2. **번호 선택**: 사용자가 권역의 번호를 입력하여 선택할 수 있도록 했습니다.
3. **결과 출력**: 선택된 권역의 여행 정보들을 **제목(Title)** 순으로 정렬하여 상세 목록으로 보여줍니다.
4. **취소 기능**: 아무것도 입력하지 않고 엔터를 치면 검색을 취소하고 메인 메뉴로 돌아갑니다.

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 종료 |

선택: 3

[권역 선택]

1) 강원권 2) 경상권 3) 수도권 4) 전라권 5) 제주권 6) 충청권

권역 번호 선택 (취소하려면 Enter): 2

[경상권 권역 목록 보기 - 정렬: 제목]

번호 | 제목 | 주소 | 전화번호

88 | 거제 수협효시공원 | 경상남도 거제시 사등면 가조서로 7 | 055-639-8340
89 | 거제 칠천량해전공원 | 경상남도 거제시 하청면 칠천로 265-39 | 055-639-8250
90 | 거제 포로수용소 유적공원 평화파크 | 경상남도 거제시 계룡로 61 (고현동) | 055-639-0625
78 | 경주 보문관광단지 | 경상북도 경주시 신평동 | 054-745-7601
79 | 고령 대가야 역사테마파크 | 경상북도 고령군 대가야읍 대가야로 1216 | 054-950-7005
...

org.scoula.travel.service.TravelService.java

```
...
public void showByDistrict() {
    List<String> districts = travelDao.getDistricts();
    if (districts.isEmpty()) {
        System.out.println("등록된 권역 정보가 없습니다.");
        return;
    }

    System.out.println("\n[권역 선택]");
    for (int i = 0; i < districts.size(); i++) {
        System.out.printf("%d) %s ", i + 1, districts.get(i));
    }
    System.out.println();

    System.out.print("권역 번호 선택 (취소하려면 Enter): ");
    String input = scanner.nextLine().trim();

    if (input.isEmpty()) return;
```

org.scoula.travel.service.TravelService.java

```
try {
    int index = Integer.parseInt(input);
    if (index < 1 || index > districts.size()) {
        System.out.println("잘못된 번호입니다.");
        return;
    }

    String selectedDistrict = districts.get(index - 1);
    List<org.scoula.travel.domain.Travel> results = travelDao.findByDistrict(selectedDistrict).stream()
        .sorted(java.util.Comparator.comparing(org.scoula.travel.domain.Travel::getTitle))
        .toList();

    System.out.printf("\n[%s 권역 목록 보기 - 정렬: 제목]\n", selectedDistrict);
    System.out.println("번호 | 제목 | 주소 | 전화번호");
    System.out.println("-----");
    for (org.scoula.travel.domain.Travel travel : results) {
        System.out.printf("%d | %s | %s | %s\n",
            travel.getNo(), travel.getTitle(),
            travel.getAddress(), travel.getPhone());
    }
    System.out.println("-----");
    System.out.printf("총 %d개의 항목이 검색되었습니다.\n", results.size());
} catch (NumberFormatException e) {
    System.out.println("숫자를 입력해주세요.");
}
}
```

✓ 상세 보기 기능 구현

○ 프롬프트

상세보기를 기능을 구현해줘.

- Travel no 값을 입력 받아 대상을 선택

✦ 상세 보기 기능을 구현했습니다.

- 사용자 입력: 목록 등에서 확인한 no(번호) 값을 입력받습니다.
- 상세 정보 출력: 해당 번호의 여행 정보가 존재할 경우 번호, 권역, 제목, 주소, 전화번호 뿐만 아니라 목록에서는 보이지 않던 상세 설명(Description)까지 전체 내용을 출력합니다.
- 예외 처리: 존재하지 않는 번호 입력 시 안내 메시지를 보여주며, 엔터만 입력할 경우 메인 메뉴로 돌아갑니다.

상세보기 할 번호(no)를 입력하세요: 2

[여행 정보 상세보기]

번호: 2

권역: 수도권

제목: 인천 월미문화의거리

주소: 인천광역시 중구 월미로 252

전화: 032-764-0842

설명:

인천의 대표적인 관광지 중의 하나인 월미도에는 월미도 문화의 거리, 월미공원, 월미테마파크 등이 있다. 월미도 문화의 거리는 보행자 전용 도로로 차량 출입시간을 제외하고는 차량의 출입이 통제되어 있다. 초상화를 그려주는 길거리 화가들이 나와 있고 앉아 쉴 수 있는 벤치가 있는 작은 쉼터 같은 거리다. 월미공원에 위치한 월미 전망대에서는 인천항 일대의 전경을 다 볼 수 있어 월미도를 간다면 한 번쯤 들러서 전망을 구경하는 것도 좋다. 놀이와 체험이 가능한 월미테마파크에는 다양한 놀이기구가 있어 연인들의 데이트 장소나 가족들의 나들이 장소로 인기 있다. 월미 달빛 음악분수는 월미 문화 거리 안에 있는 분수로 크지는 않지만 월미도의 볼거리 중 하나이다. 5월1일부터 10월 31일까지 진행하고 매주 월요일은 정기점검을 한다. 음악분수는 음악에 맞춰 불줄기들이 춤을 추는데 물과 음악이 조합되어 멋진 장관을 이룬다. 저녁이 되면 음악에 따라 6~7가지 빛의 분수 쇼가 펼쳐진다. 그러므로 낮에 감상하는 것보다는 밤에 보는 모습이 더욱더 멋지다.

org.scoula.travel.service.TravelService.java

```
public void showDetail() {
    System.out.print("상세보기 할 번호(no)를 입력하세요: ");
    String input = scanner.nextLine().trim();

    if (input.isEmpty()) return;

    try {
        int no = Integer.parseInt(input);
        java.util.Optional<org.scoula.travel.domain.Travel> travelOpt = travelDao.findByNo(no);

        if (travelOpt.isPresent()) {
            org.scoula.travel.domain.Travel travel = travelOpt.get();
            System.out.println("\n[여행 정보 상세보기]");
            System.out.println("-----");
            System.out.printf("번호: %d\n", travel.getNo());
            System.out.printf("구역: %s\n", travel.getDistrict());
            System.out.printf("제목: %s\n", travel.getTitle());
            System.out.printf("주소: %s\n", travel.getAddress());
            System.out.printf("전화: %s\n", travel.getPhone());
            System.out.println("-----");
            System.out.println("설명:");
            System.out.println(travel.getDescription());
            System.out.println("-----");
        } else {
            System.out.printf("번호 %d에 해당하는 정보를 찾을 수 없습니다.\n", no);
        }
    }
```

org.scoula.travel.service.TravelService.java

```
    } catch (NumberFormatException e) {  
        System.out.println("숫자를 입력해주세요.");  
    }  
}
```


✓ 기능 추가

○ 프롬프트

5번 메뉴로 검색 기능을 추가해줘.

- 검색어를 입력 받음
- 제목(title)과 설명(description)을 대상으로 검색
- 검색결과를 권역, 제목 순으로 정렬

✦ 검색 기능을 5번 메뉴로 추가하고 구현을 완료했습니다.

주요 변경 사항

1. 메뉴 구성 업데이트: `TravelApp`에서 5번에 "검색"을 추가하고, 기존 "종료"를 6번으로 이동했습니다.
2. `showSearch()` 구현:
 - 사용자로부터 검색어를 입력받습니다.
 - 제목(`Title`)과 설명(`Description`) 중 하나라도 검색어가 포함된 항목을 찾아냅니다.
 - 검색 결과를 권역(`District`) → 제목(`Title`) 순으로 정렬하여 출력합니다.
 - 검색된 총 건수를 하단에 표시합니다.

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 검색 | 6. 종료 |

선택: 5

검색어(제목/설명 포함)를 입력하세요: 해수욕장

['해수욕장' 검색 결과 - 정렬: 권역/제목]

번호 | 권역 | 제목 | 주소 | 전화번호

106 | 강원권 | 강릉 정동진 모래시계공원 | 강원도 강릉시 강동면 헌화로 990-1 | 033-640-4533
107 | 강원권 | 동해 망상해수욕장 | 강원도 동해시 동해대로 6270-10 (망상동) | 033-530-2800
99 | 강원권 | 속초 해수욕장관광지 | 강원도 속초시 해오름로 190 (조양동) | 033-639-2027
76 | 경상권 | 부산 해운대해수욕장 | 부산광역시 해운대구 해운대해변로 264 (우동) | 051-749-5700
4 | 수도권 | 인천 하나개해수욕장 | 인천광역시 중구 무의동 산189번지 (무의동) | 032-751-8833
45 | 전라권 | 부안 모항해수욕장 | 전라북도 부안군 변산면 모항길 22 | 063-580-4413
44 | 전라권 | 부안 변산해수욕장 | 전라북도 부안군 변산면 변산로 2076 | 063-583-6951
71 | 전라권 | 완도 신지 명사십리해수욕장 | 전라남도 완도군 신지면 | 061-550-6921
14 | 충청권 | 보령 대천해수욕장 | 충청남도 보령시 머드로 123 | 041-933-7051

총 9개의 항목이 검색되었습니다.

1. 전체 목록 | 2. 페이지별 목록 | 3. 권역별 목록 | 4. 상세보기 | 5. 검색 | 6. 종료 |

선택:

org.scoula.travel.TravelApp.java

```
package org.scoula.travel;

import org.scoula.cli.App;
import org.scoula.travel.service.TravelService;

public class TravelApp extends App {
    private final TravelService travelService = new TravelService();

    @Override
    public void init() {
        register("전체 목록", travelService::showAll);
        register("페이지별 목록", travelService::showPage);
        register("권역별 목록", travelService::showByDistrict);
        register("상세보기", travelService::showDetail);
        register("검색", travelService::showSearch);
        register("종료", travelService::exit);
    }

    public static void main(String[] args) {
        new TravelApp().run();
    }
}
```

org.scoula.travel.service.TravelService.java

```
public void showSearch() {
    System.out.print("검색어(제목/설명 포함)를 입력하세요: ");
    String keyword = scanner.nextLine().trim();

    if (keyword.isEmpty()) {
        System.out.println("검색어를 입력해주세요.");
        return;
    }

    List<org.scoula.travel.domain.Travel> results = travelDao.search(keyword).stream()
        .sorted(java.util.Comparator.comparing(org.scoula.travel.domain.Travel::getDistrict)
            .thenComparing(org.scoula.travel.domain.Travel::getTitle))
        .toList();
}
```

org.scoula.travel.service.TravelService.java

```
if (results.isEmpty()) {
    System.out.printf("'s' 검색 결과가 없습니다.%n", keyword);
} else {
    System.out.printf("\n['s' 검색 결과 - 정렬: 권역/제목]%n", keyword);
    System.out.println("번호 | 권역 | 제목 | 주소 | 전화번호");
    System.out.println("-----");
    for (org.scoula.travel.domain.Travel travel : results) {
        System.out.printf("%d | %s | %s | %s | %s%n",
            travel.getNo(), travel.getDistrict(), travel.getTitle(),
            travel.getAddress(), travel.getPhone());
    }
    System.out.println("-----");
    System.out.printf("총 %d개의 항목이 검색되었습니다.%n", results.size());
}
}
```

✓ 클래스 다이어그램 그리기

○ 프롬프트

- org.scoula.cli 패키지와 org.scoula.travel 패키지를 클래스 다이어그램으로 그려서 class_diagram.md 파일로 만들어 줘.

✦ 클래스 다이어그램을 포함한 `class_diagram.md` 파일을 생성했습니다. 이 파일에서 프레임워크와 애플리케이션 패키지 간의 관계를 상세히 확인할 수 있습니다.

class_diagram.md

CLI Framework and Travel Application Class Diagram

```
```mermaid
classDiagram
 package org.scoula.cli {
 class Command {
 <<interface>>
 +execute() void
 }

 class Menu {
 -items: List~MenuItem~
 +add(MenuItem) void
 +print() void
 +getCommand(int index) Command
 }

 class MenuItem {
 -title: String
 -command: Command
 +getTitle() String
 +getCommand() Command
 }
 }
```

## class\_diagram.md

```
...
%% Relationships
App "1" *-- "1" Menu : contains
Menu "1" *-- "n" MenuItem : contains
MenuItem "1" o-- "1" Command : references
TravelApp --|> App : extends
TravelApp "1" o-- "1" TravelService : uses
TravelService "1" o-- "1" TravelDao : uses
TravelDao "1" *-- "n" Travel : manages
TravelService ..> Travel : uses
...
```

### ### 다이어그램 설명

1. **\*\*`org.scoula.cli` (Framework)\*\*:**
  - **\*\*`Command`\*\*:** 실행 로직을 추상화한 인터페이스입니다.
  - **\*\*`App`\*\*:** 프레임워크의 엔진 역할을 하는 추상 클래스로, `Menu`를 관리하고 실행 루프를 제공합니다.
  - **\*\*`Menu` & `MenuItem`\*\*:** 명령어들을 제목과 함께 관리하고 인덱스 기반으로 접근할 수 있게 합니다.
2. **\*\*`org.scoula.travel` (Application)\*\*:**
  - **\*\*`TravelApp`\*\*:** 프레임워크(`App`)를 상속받아 실제 메뉴 항목(`TravelService` 메서드들)을 등록하고 실행합니다.
  - **\*\*`TravelService`\*\*:** 사용자의 요구사항(비즈니스 로직)을 처리하며, `TravelDao`를 통해 데이터를 조작하고 결과를 출력합니다.
  - **\*\*`TravelDao`\*\*:** `travel.csv` 파일에서 데이터를 로드하고, 검색, 페이징, 필터링 등 데이터 접근 기능을 담당합니다.
  - **\*\*`Travel`\*\*:** 여행 정보를 담는 도메인 모델(VO/DTO)입니다.



## ✓ 이미지로 그리기

- <https://gemini.google.com/>

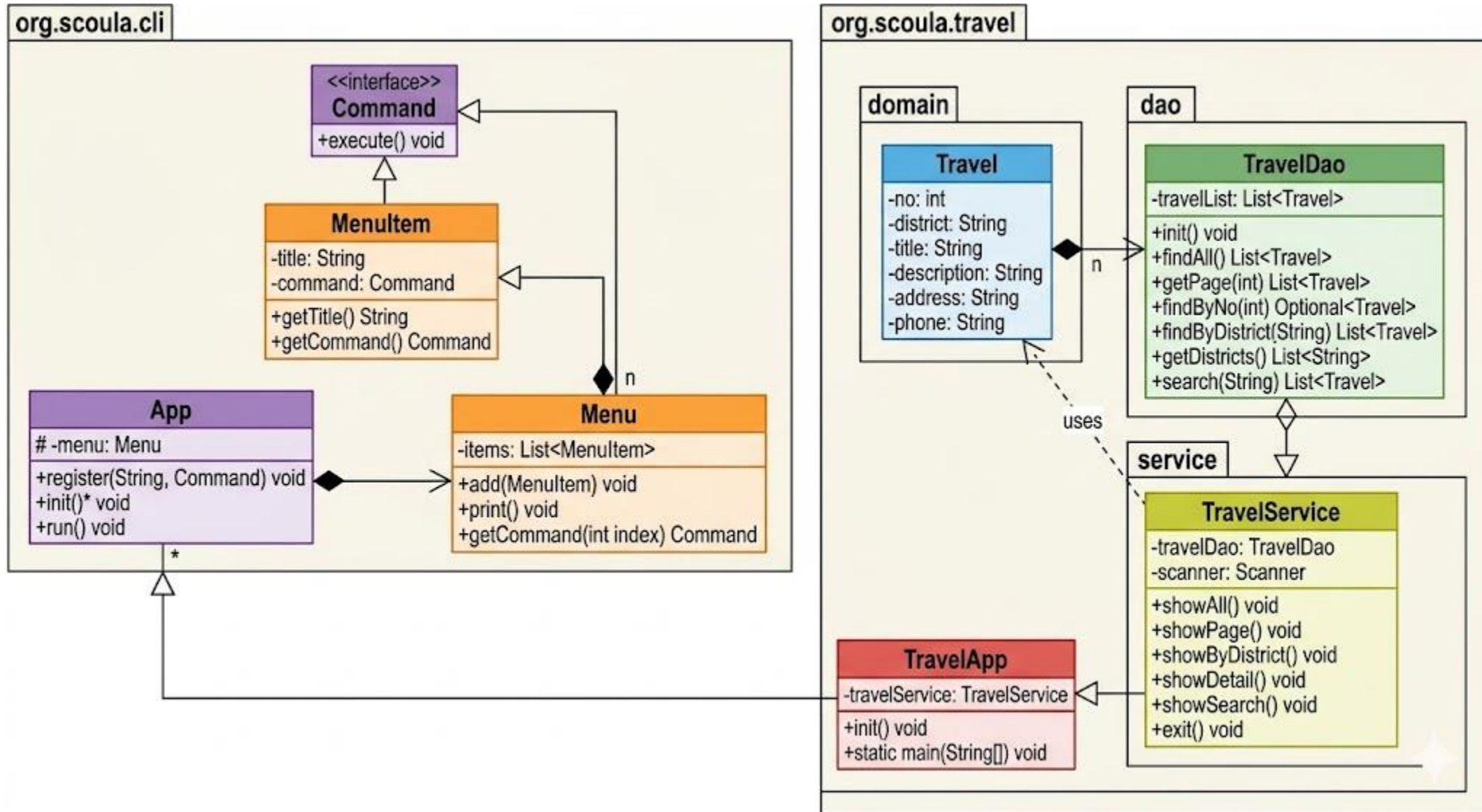
- 프로프트

```
```mermaid 코드
```

```
...  
```
```

클래스 다이어그램을 이미지로 그려줘

클래스 다이어그램을 분석해줘.



## ☒ 주요 설계 구조 분석

### 1. CLI 프레임워크 계층 (org.scoula.cli)

- **Command 패턴:** Command 인터페이스를 통해 실행 로직을 캡슐화하여 메뉴와 기능을 분리했습니다.
- **컴포지션 구조:** App은 Menu를 가지고 있고, Menu는 여러 개의 MenuItem을 포함하는 전형적인 계층 구조입니다.
- **추상화:** App 클래스를 추상화하여, 새로운 CLI 프로그램을 만들 때 init()만 구현하면 되도록 확장성을 높였습니다.

### 2. 비즈니스 도메인 계층 (org.scoula.travel)

- **Layered Architecture:** \* App (Controller 역할): 프로그램의 진입점 및 메뉴 구성.
  - **Service:** 사용자 입력을 받고 로직을 처리하는 비즈니스 계층.
  - **DAO (Data Access Object):** 데이터(Travel 객체 리스트)에 접근하고 필터링하는 데이터 전용 계층.
  - **Domain:** 여행 정보를 담는 순수 객체(Value Object).

이 구조는 특히 **\*\*관심사의 분리(SoC)\*\***가 잘 되어 있어, 나중에 데이터 소스가 메모리(List)에서 데이터베이스(JDBC)로 바뀌더라도 TravelDao 코드만 수정하면 서비스나 앱 로직에는 영향을 주지 않는 유연한 구조입니다.